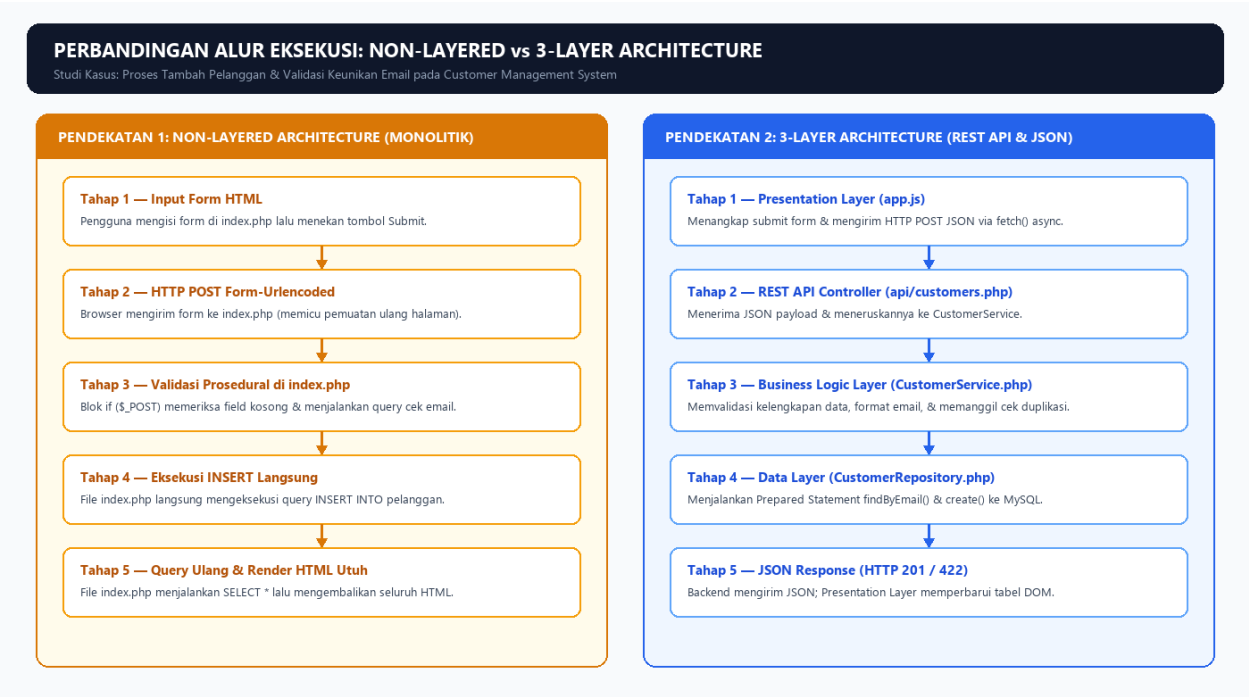
LAPORAN TUGAS KELOMPOK PEMROGRAMAN WEB

ANALISIS DAN IMPLEMENTASI PERBANDINGAN LAYERED VS NON-LAYERED WEB APPLICATION ARCHITECTURE

Studi Kasus: Customer Management System (PHP Native, REST API JSON, & MySQL)



Judul Tugas	Analisis dan Implementasi Perbandingan Layered vs Non-Layered Web Application Architecture
Studi Kasus	Customer Management System (Pengelolaan Data Pelanggan Perusahaan)
6 Fitur Wajib	1. Tambah Pelanggan, 2. Tampil Pelanggan, 3. Edit Pelanggan, 4. Hapus Pelanggan, 5. Pencarian (Nama/Email), 6. Validasi (Kelengkapan & Email Unik)
Aplikasi 1 (Non-Layered)	Tanpa Pemisahan Layer Tegas (bisa 1/beberapa file tercampur; pada studi kasus ini diimplementasikan dalam 1 file 1-non-layered-app/index.php)
Aplikasi 2 (3-Layer)	Pemisahan 3 Folder: presentation/ (HTML/CSS/JS), business-logic/ (Service & REST API JSON), data-access/ (PDO MySQL)
Database SQL	database/customer_management.sql (Database MySQL customer_management, Tabel pelanggan + 5 Data Awal)

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan aplikasi berbasis web, terdapat berbagai pendekatan arsitektur yang dapat dipilih oleh pengembang untuk menyusun kode tampilan antarmuka (*frontend*), logika proses bisnis (*business logic*), serta pengelolaan database (*data access*). Dua pendekatan utama yang sering dipelajari dan dibandingkan dalam rekayasa perangkat lunak web adalah **Non-Layered Architecture** (arsitektur tanpa pemisahan layer secara tegas) dan **Layered Architecture** (khususnya **3-Layer Web Application Architecture**).

Pendekatan Pertama (Non-Layered Architecture) merupakan pendekatan di mana aplikasi dikembangkan tanpa pemisahan layer secara tegas; kode tampilan, proses bisnis, dan akses database **dapat berada dalam satu file atau beberapa file yang belum memiliki pemisahan tanggung jawab yang jelas** (pada implementasi studi kasus proyek ini ditempatkan dalam satu file index.php). Sementara itu, **Pendekatan Kedua (3-Layer Architecture)** memisahkan tanggung jawab aplikasi secara tegas ke dalam tiga lapisan yang berbeda—yaitu *Presentation Layer*, *Business Logic Layer*, dan *Data Layer*—yang saling berkomunikasi melalui protokol standar seperti REST API dan JSON. Kedua pendekatan ini masing-masing memiliki karakteristik, cara kerja, serta kesesuaian tersendiri tergantung pada skala dan kebutuhan proyek. Oleh karena itu, melalui studi kasus **Customer Management System** ini, kedua arsitektur diimplementasikan secara langsung untuk diuji dan dibandingkan secara objektif.

1.2 Rumusan Masalah

1. Apa pengertian, karakteristik, dan cara kerja dari **Non-Layered Architecture** maupun **3-Layer Web Application Architecture**?
2. Bagaimana cara mengimplementasikan studi kasus **Customer Management System** (dengan 6 fitur wajib) menggunakan pendekatan Non-Layered dibandingkan dengan pendekatan 3-Layer (3 folder terpisah dengan komunikasi REST API & JSON)?
3. Bagaimana perbedaan struktur pemisahan file/folder serta alur proses *request-response* pada kedua pendekatan arsitektur tersebut?
4. Berdasarkan hasil pengujian pada berbagai unsur (struktur file/folder, kecepatan pembuatan awal, pengerjaan individu vs tim, kemudahan penelusuran kesalahan/bug, kecepatan respons UI, keamanan edit desain, penambahan fitur, dan reusabilitas API), bagaimana perbandingan hasil kedua arsitektur dan kapan masing-masing lebih unggul?

1.3 Tujuan Penulisan & Implementasi

- Memahami konsep dasar, struktur, dan cara kerja **Non-Layered Architecture** serta **3-Layer Web Application Architecture** secara objektif.
- Membangun dua aplikasi web **Customer Management System** menggunakan PHP Native dan MySQL dengan 6 fitur wajib yang identik pada kedua arsitektur.
- Menyajikan perbandingan visual melalui **Diagram Arsitektur (Pemisahan File/Folder)**, **Screenshot Code** kedua konsep, serta **Screenshot Tampilan Web**.
- Menyusun **Satu Tabel Utama Hasil Pengujian dan Perbandingan** untuk menganalisis keunggulan dan alasan pemilihan masing-masing arsitektur secara objektif.

BAB 2. LANDASAN TEORI & KONSEP DASAR KEDUA ARSITEKTUR

2.1 Konsep Pendekatan Pertama: Non-Layered Architecture (Halaman 14–16 & 18 PDF)

Non-Layered Architecture (Aplikasi Tanpa 3-Layer) merupakan pendekatan pengembangan aplikasi web yang biasa disebut sebagai **arsitektur monolitik sederhana** (*monolithic application*), yaitu aplikasi yang dikembangkan **tanpa pemisahan layer secara tegas**. Sebagaimana dijelaskan pada Halaman 16 dan 18 PDF, kode tampilan (*frontend*), proses bisnis (validasi), dan akses database (query SQL) **dapat berada dalam satu**

file atau beberapa file yang belum memiliki pemisahan tanggung jawab (layer) yang jelas. Pada studi kasus sederhana (sesuai Halaman 14 & 20 PDF), seluruh proses tersebut ditempatkan secara terpadu di dalam **satu file PHP (index.php)** mulai dari: (1) Menampilkan formulir, (2) Menerima input pengguna, (3) Memvalidasi data, (4) Mengakses database MySQL, hingga (5) Menampilkan hasil.

Catatan Penting (Halaman 16 PDF): *Aplikasi tanpa 3-layer ini bukan berarti sama sekali tidak memiliki tiga tanggung jawab tersebut. Fungsi tampilan, proses bisnis, dan akses data tetap ada di dalamnya, tetapi tidak dipisahkan menjadi layer yang jelas.*

2.2 Konsep Pendekatan Kedua: 3-Layer Web Application Architecture (Halaman 2–11 PDF)

3-Layer Web Application Architecture adalah konsep pengembangan aplikasi web dengan membagi sistem menjadi tiga lapisan (*layer*) yang memiliki tugas dan tanggung jawab berbeda:

1. **Presentation Layer (Lapisan Antarmuka / Frontend):** Bagian aplikasi yang berinteraksi langsung dengan pengguna, bertugas menampilkan informasi dan menerima masukan dari pengguna menggunakan **HTML**, **CSS**, dan **JavaScript**. *Intinya: Presentation Layer bertanggung jawab terhadap apa yang dilihat dan digunakan oleh pengguna.*
2. **Application / Business Logic Layer (Lapisan Logika Bisnis / Backend):** Bagian yang menangani proses bisnis atau aturan utama aplikasi, menerima permintaan dari frontend, memproses/memvalidasinya, kemudian berkomunikasi dengan database jika diperlukan. *Intinya: Business Logic Layer bertanggung jawab terhadap bagaimana aplikasi bekerja dan mengambil keputusan.*
3. **Data Layer (Lapisan Pengelolaan Data / Database):** Bagian yang bertugas menyimpan, mengambil, memperbarui, dan menghapus data di dalam database (seperti **MySQL**). *Intinya: Data Layer bertanggung jawab terhadap penyimpanan dan pengelolaan data aplikasi.*

Aspek	Presentation Layer	Business Logic Layer	Data Layer
Fungsi Utama	Tampilan & Interaksi User	Proses & Aturan Bisnis	Penyimpanan & Query Data
Berinteraksi Dengan	Pengguna (Browser)	Frontend & Data Layer	Business Logic Layer (Backend)
Teknologi Proyek Ini	HTML5, CSS3, JavaScript (Fetch)	PHP Native (Service & REST API)	PDO PHP & Database MySQL
Folder pada Proyek	2-layered-app/presentation/	2-layered-app/business-logic/	2-layered-app/data-access/
Tanggung Jawab	Menampilkan & menerima data	Memvalidasi & mengolah data	Menyimpan & mengambil data SQL

Tabel 2.1 Pembagian Tanggung Jawab Ketiga Layer pada Arsitektur 3-Layer (Sesuai Halaman 11 PDF)

2.3 Karakteristik Arsitektur Non-Layered vs 3-Layer (Halaman 12–16 PDF)

Kedua pendekatan arsitektur memiliki cara kerja dan karakteristik struktural yang berbeda. Pada **Non-Layered Architecture** (Halaman 15 PDF), alur kerja berjalan dalam 6 tahap langsung di satu file: (1) Pengguna mengisi formulir, (2) Browser mengirim data ke index.php via HTTP POST, (3) PHP memvalidasi dan menjalankan query SQL, (4) MySQL menyimpan data, (5) PHP menjalankan SELECT data terbaru, dan (6) Browser menampilkan kembali halaman HTML utuh.

Sementara itu, pada **3-Layer Architecture** (Halaman 10 & 12 PDF), sistem dirancang berdasarkan lima karakteristik arsitektural: (1) **Separation of Concerns** (pemisahan tanggung jawab tiap layer), (2) **Maintainability** (kemudahan pemeliharaan tampilan & database secara terpisah), (3) **Scalability** (pengembangan tiap layer sesuai kebutuhan), (4) **Security** (isolasi akses database di Data Layer), serta (5) **Reusability** (penggunaan ulang API oleh berbagai platform). Perbandingan karakteristik dasar kedua konsep tersebut dirangkum pada Tabel 2.2:

Aspek Karakteristik	Pendekatan 1: Tanpa 3-Layer (Non-Layered)	Pendekatan 2: Dengan 3-Layer Architecture
Struktur Kode	Sederhana (belum memiliki pemisahan layer yang tegas)	Terpisah secara tegas ke dalam 3 layer
File Program	Bisa satu file atau beberapa file tanpa batas layer (di sini 1 file index.php)	Dibagi ke beberapa komponen/folder sesuai layer
Bagian Frontend	Menyatu langsung dengan kode PHP	Terpisah di folder presentation/
Bagian Business Logic	Menyatu dengan kode aplikasi & tampilan	Dipisahkan di folder business-logic/
Bagian Database Access	Langsung ditulis di dalam file PHP utama	Melalui komponen khusus di folder data-access/
Karakteristik Pemeliharaan	Praktis saat kecil, lebih rumit jika fitur sangat kompleks	Lebih terstruktur karena perubahan terlokalisasi per layer
Karakteristik Pengembangan	Sangat cepat untuk pembuatan aplikasi kecil / individu	Lebih terorganisasi untuk pengembangan tim & jangka panjang
Karakteristik Pengujian	Diuji secara keseluruhan dalam satu file terpadu	Dapat diuji secara terpisah pada masing-masing layer

Tabel 2.2 Perbandingan Karakteristik Dasar Tanpa 3-Layer vs Dengan 3-Layer (Sesuai Halaman 16 PDF)

BAB 3. STUDI KASUS: CUSTOMER MANAGEMENT SYSTEM & DATABASE

3.1 Spesifikasi 6 Fitur Wajib (Halaman 18–19 PDF)

Sesuai instruksi pada Halaman 18–19 PDF, studi kasus yang dibangun adalah **Customer Management System**, yaitu aplikasi web untuk mengelola data pelanggan sebuah perusahaan. Kedua aplikasi (baik Non-Layered maupun 3-Layer) wajib mengimplementasikan **6 fitur utama** berikut:

No	Nama Fitur Wajib	Keterangan Fungsi	Implementasi pada Proyek
1	**Tambah Pelanggan**	Memasukkan data pelanggan baru	Input nama, email, telepon, dan alamat ke tabel pelanggan
2	**Tampil Pelanggan**	Menampilkan seluruh data pelanggan	Menampilkan tabel daftar pelanggan diurutkan dari ID terbaru
3	**Edit Pelanggan**	Mengubah informasi pelanggan	Mengambil data berdasarkan ID lalu memperbarui baris di MySQL
4	**Hapus Pelanggan**	Menghapus data pelanggan	Menghapus data pelanggan berdasarkan ID dengan konfirmasi
5	**Pencarian**	Mencari berdasarkan nama atau email	Query pencarian menggunakan WHERE nama LIKE ? OR email LIKE ?
6	**Validasi**	Memeriksa kelengkapan & keunikan email	Menolak kolom kosong, format email salah, atau email yang sudah terdaftar

Tabel 3.1 Enam Fitur Wajib Studi Kasus Customer Management System (Sesuai Halaman 19 PDF)

3.2 Struktur Tabel Database MySQL (File: database/customer_management.sql)

Kedua aplikasi menggunakan database MySQL bernama customer_management dengan tabel pelanggan yang sudah dilengkapi 5 data pelanggan awal (*seed data*):

Nama Kolom	Tipe Data	Constraint / Indeks	Keterangan
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	ID unik setiap pelanggan
nama	VARCHAR(100)	NOT NULL, INDEX	Nama lengkap pelanggan perusahaan
email	VARCHAR(100)	NOT NULL, UNIQUE	Alamat email pelanggan (wajib unik / tidak boleh sama)
telepon	VARCHAR(25)	NOT NULL	Nomor telepon aktif pelanggan
alamat	TEXT	NOT NULL	Alamat lengkap pelanggan
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Waktu pencatatan data pelanggan

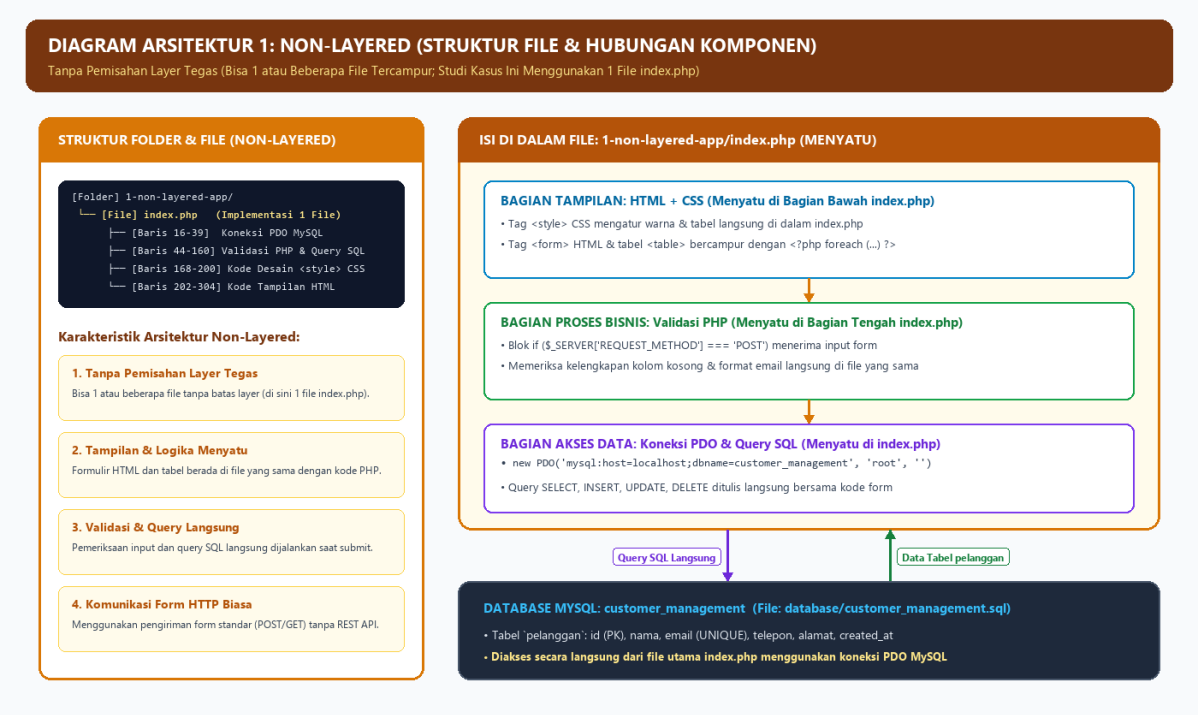
Tabel 3.2 Struktur Tabel pelanggan pada File database/customer_management.sql

3.3 Ketentuan Teknis Kedua Aplikasi (Halaman 20–21 PDF)

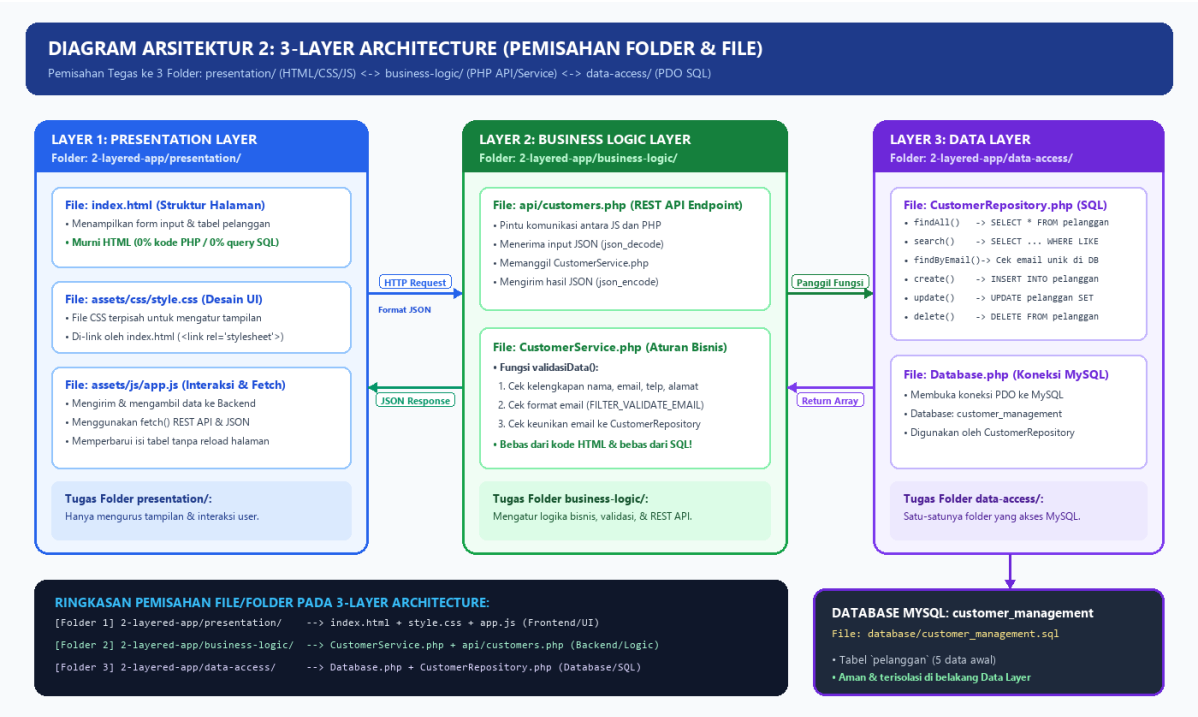
- **Aplikasi Pertama (Non-Layered Architecture — Halaman 20 PDF):** Dibangun menggunakan PHP Native dan MySQL di mana tampilan HTML digabungkan dengan PHP, proses bisnis ditempatkan dalam file yang sama dengan tampilan, koneksi database dan query ditulis langsung dalam file aplikasi (1-non-layered-app/index.php), tidak menggunakan REST API, dan seluruh fitur CRUD berjalan.
- **Aplikasi Kedua (3-Layer Architecture — Halaman 21 PDF):** Membangun kembali aplikasi yang sama menggunakan PHP Native (non-framework) dengan memisahkan tanggung jawab menjadi 3 folder: **Presentation Layer** (presentation/) menangani tampilan & interaksi user, **Business Logic Layer** (business-logic/) menangani proses & aturan bisnis, serta **Data Layer** (data-access/) menangani komunikasi dengan database MySQL. Komunikasi antara frontend dan backend wajib menggunakan **REST API dan JSON**.

BAB 4. DIAGRAM ARSITEKTUR (PEMISAHAN FILE/FOLDER & HUBUNGAN KOMPONEN)

Sesuai arahan dosen, diagram arsitektur dirancang untuk memperlihatkan secara jelas **struktur pemisahan file/folder** serta bagaimana komponen **HTML, CSS, JavaScript, PHP, dan SQL** saling berhubungan pada kedua pendekatan:



Gambar 4.1 Diagram Arsitektur Non-Layered: Seluruh Kode (HTML, CSS, Validasi PHP, & Query SQL) Menyatu dalam 1 File index.php



Gambar 4.2 Diagram Arsitektur 3-Layer: Pemisahan Tegas ke dalam 3 Folder Layer (presentation/, business-logic/, data-access/)

BAB 5. SCREENSHOT CODE KEDUA KONSEP ARSITEKTUR

Untuk membuktikan perbedaan konsep secara nyata pada level kode sumber (*source code*), berikut adalah perbandingan langsung potongan kode utama pada aplikasi **Non-Layered** (1 file monolitik) dan **3-Layer Architecture** (3 folder terpisah):

SCREENSHOT CODE 1: KONSEP NON-LAYERED ARCHITECTURE (1 FILE MONOLITIK)
File: 1-non-layered-app/index.php — Koneksi Database. Validasi Proses Bisnis. Query SQL, CSS, dan HTML Menyatu dalam 1 File

1-non-layered-app/index.php (Seluruh Kode Menyatu dalam 1 File) — NON-LAYERED (MONOLITIK)

```php
// ============================================================================
// BAGIAN 1: KONEKSI LANGSUNG KE DATABASE MYSQL (Di dalam index.php)
// ============================================================================
$koneksi = new PDO("mysql:host=localhost;dbname=customer_management", "root", "");

// ============================================================================
// BAGIAN 2: PROSES BISNIS (VALIDASI) & QUERY CRUD (Menyatu di index.php)
// ============================================================================
if (isset($_GET['hapus'])) {
    $stmt = $koneksi->prepare("DELETE FROM pelanggan WHERE id = ?");
    $stmt->execute([(int) $_GET['hapus']]);
}
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // A. Validasi Kelengkapan Data
    if ($nama === '' || $email === '' || $telepon === '' || $alamat === '') {
        $pesan_error[] = "Semua kolom pelanggan wajib diisi.";
    }
    // B. Validasi Keunikan Email (Langsung query SQL di file tampilan)
    $stmtCek = $koneksi->prepare("SELECT id FROM pelanggan WHERE LOWER(email) = ?");
    $stmtCek->execute([$email]);
    if ($stmtCek->fetch()) {
        $pesan_error[] = "Email sudah terdaftar di database!";
    }
    // C. Eksekusi Query INSERT / UPDATE Langsung
    $stmt = $koneksi->prepare("INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES (?,?,?,?)");
    $stmt->execute([$nama, $email, $telepon, $alamat]);
}
// Query Tampil & Pencarian Pelanggan
$daftar_pelanggan = $koneksi->query("SELECT * FROM pelanggan ORDER BY id DESC")->fetchAll();
?>
<!-- ============================================================================
     BAGIAN 3: TAMPILAN HTML & CSS (Menyatu langsung di bawah kode PHP)
     ============================================================================ -->
<!DOCTYPE html><html lang="id"><head><style> /* Kode CSS di sini */ </style></head>
```

Gambar 5.1 Screenshot Code Non-Layered Architecture (1-non-layered-app/index.php): Koneksi PDO, Validasi, Query SQL, CSS, dan HTML Menyatu

SCREENSHOT CODE 2: KONSEP 3-LAYER ARCHITECTURE (TERPISAH 3 FOLDER LAYER)
Pemisahan Tegas: Presentation Layer (app.js) <-> Business Logic Layer (CustomerService.php & api) <-> Data Layer (CustomerRepository.php)

presentation/app.js — LAYER 1: PRESENTATION

```javascript
// Folder: 2-layered-app/presentation/
// Murni Frontend (Tanpa PHP & Tanpa SQL)
const URL_API = '../business-logic/api/customers.php';

// 1. Ambil Data via GET REST API (JSON)
async function muatDataPelanggan(kataKunci = '') {
  const response = await fetch(url);
  const hasil = await response.json();
  tbody.innerHTML = hasil.data.map(...);
}

// 2. Kirim Data via POST/PUT JSON
const response = await fetch(url, {
  method: idEdit ? 'PUT' : 'POST',
  headers: {'Content-Type':'application/json'},
  body: JSON.stringify(dataKirim)
});
const hasil = await response.json();
```

CustomerService.php — LAYER 2: BUSINESS LOGIC

```php
// Folder: 2-layered-app/business-logic/
// Aturan Bisnis, Validasi, & REST API JSON
class CustomerService {
  public function createCustomer($input) {
    $data = $this->rapikanInput($input);
    $errors = $this->validasiData($data);
    if (!empty($errors)) {
      return ['status_code' => 422,
              'success' => false,
              'errors' => $errors];
    }
    $baru = $this->repo->create($data);
    return ['status_code' => 201,
            'success' => true,
            'data' => $baru];
  }
  // Validasi kelengkapan & email unik
  private function validasiData($data) {...}
}
```

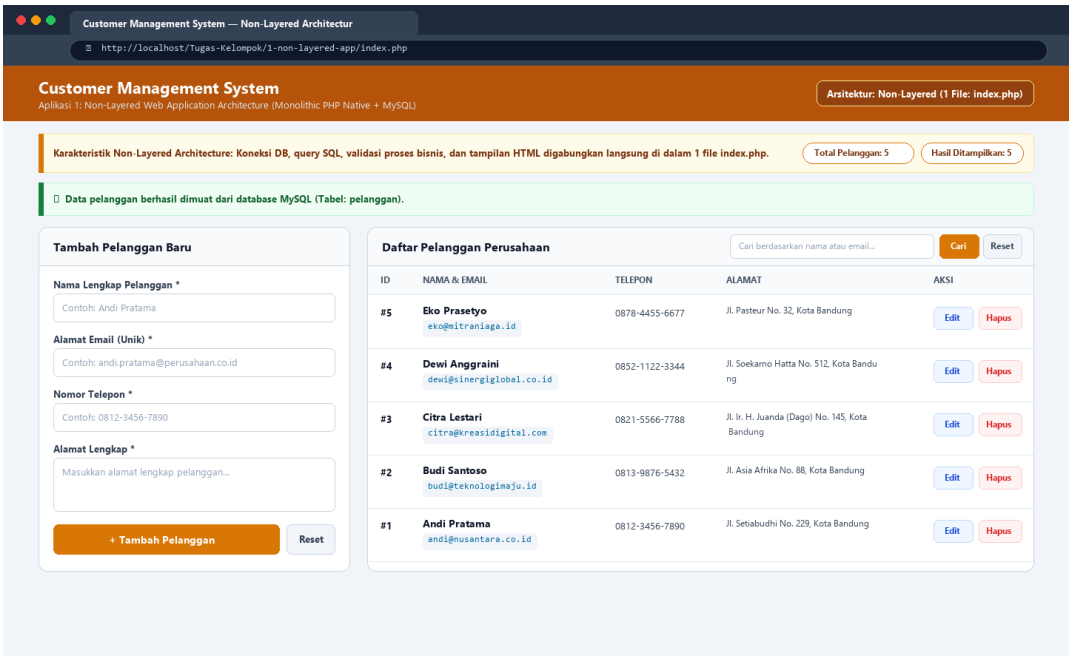
CustomerRepository.php — LAYER 3: DATA LAYER

```php
// Folder: 2-layered-app/data-access/
// Khusus Koneksi PDO & Query SQL MySQL
class CustomerRepository {
  public function findAll() {
    return $this->db->query(
      'SELECT * FROM pelanggan ORDER BY id DESC'
    )->fetchAll();
  }
  public function findByEmail($email) {
    $stmt = $this->db->prepare(
      'SELECT * FROM pelanggan WHERE email = ?'
    );
  }
  public function create($data) {
    $stmt = $this->db->prepare(
      'INSERT INTO pelanggan VALUES (?,?,?,?)'
    );
    $stmt->execute([...]);
  }
}
```

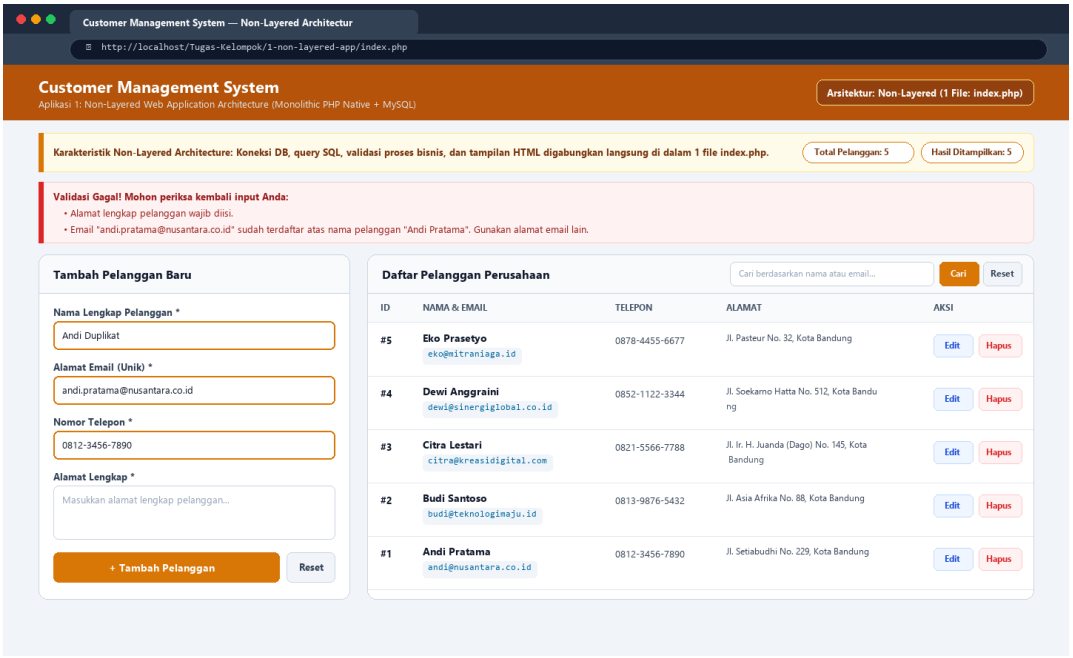
Gambar 5.2 Screenshot Code 3-Layer Architecture: Pemisahan Tegas antara app.js (Presentation), CustomerService.php (Logic), dan CustomerRepository.php (Data)

BAB 6. SCREENSHOT TAMPILAN WEB & PERBANDINGAN ALUR PROSES

6.1 Screenshot Tampilan Aplikasi 1 (Non-Layered Architecture)

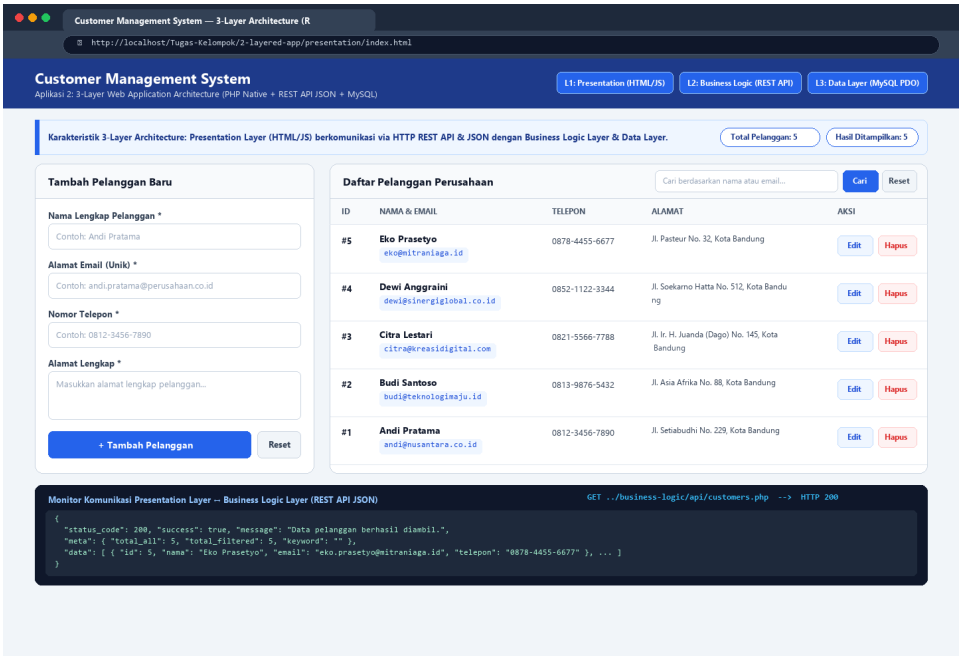


Gambar 6.1 Tampilan Utama & Tabel Daftar Pelanggan pada Aplikasi 1 (1-non-layered-app/index.php)

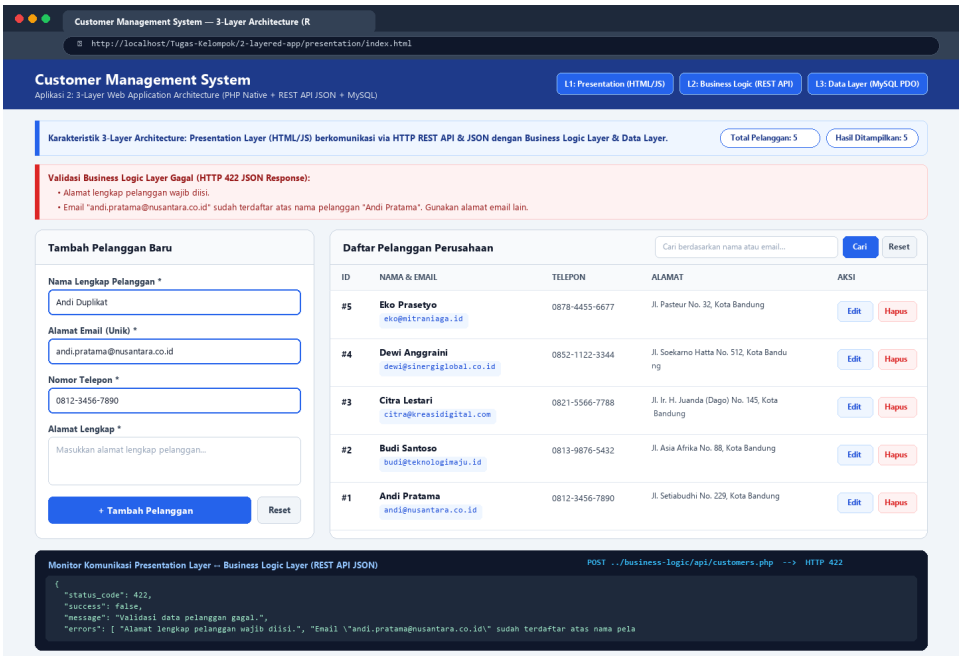


Gambar 6.2 Tampilan Pesan Validasi Kelengkapan Data & Email Unik pada Aplikasi 1

6.2 Screenshot Tampilan Aplikasi 2 (3-Layer Architecture dengan REST API & JSON)



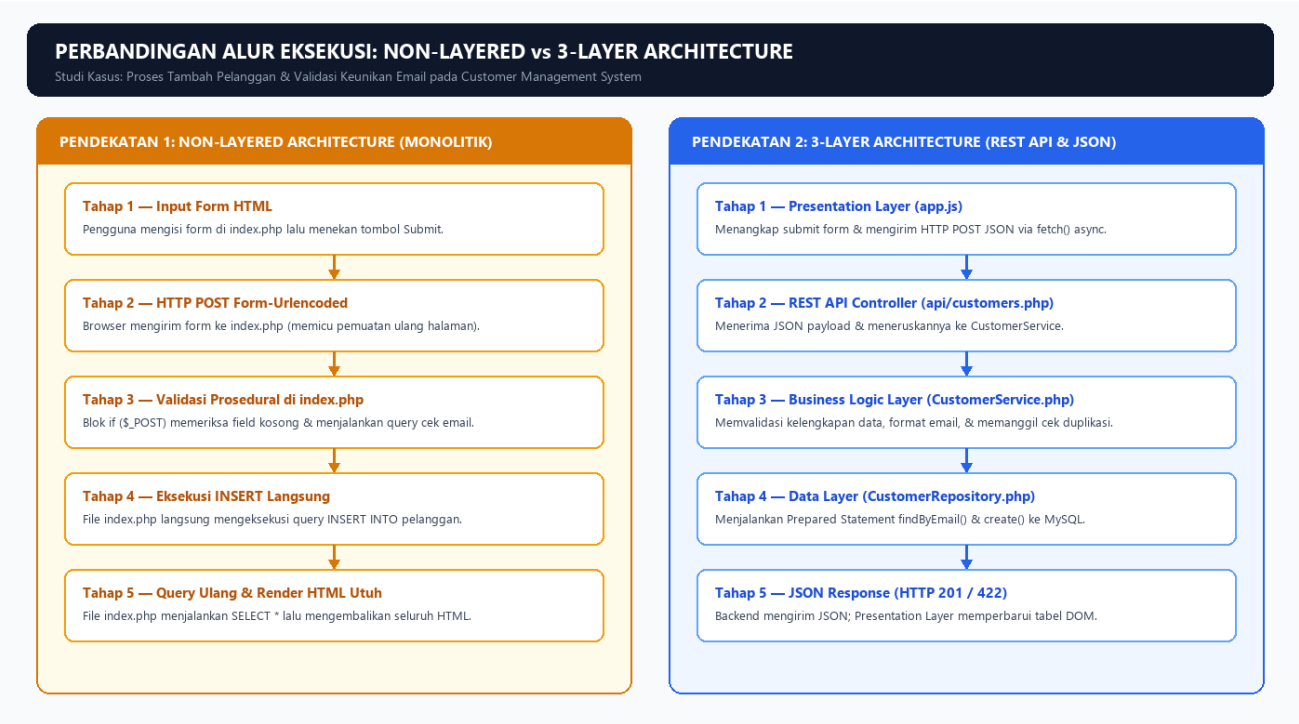
Gambar 6.3 Tampilan Utama Aplikasi 2 (3-Layer Architecture) beserta Panel Monitor Respons JSON (HTTP 200 OK)



Gambar 6.4 Tampilan Validasi Business Logic Layer (HTTP 422 Unprocessable Entity JSON) pada Aplikasi 2

6.3 Perbandingan Alur Proses Request & Response Kedua Aplikasi

Setelah melihat demo dan tampilan kedua aplikasi, perbedaan cara kerja di balik layar antara pengiriman form tradisional (*Full Page Reload*) pada Non-Layered dan komunikasi asinkron **REST API & JSON** pada 3-Layer dapat diringkas pada diagram berikut:



Gambar 6.5 Diagram Perbandingan Alur Proses: 6 Tahap Non-Layered (Full Page Reload) vs 7 Tahap 3-Layer (REST API & JSON)

BAB 7. TABEL HASIL PENGUJIAN DAN PERBANDINGAN AKHIR

Sesuai arahan dosen pengampu, berikut adalah **Satu Tabel Utama Hasil Pengujian dan Perbandingan** yang merangkum unsur-unsur apa saja yang diuji coba, bagaimana hasilnya pada pendekatan **Tanpa Layer (Non-Layered)** dibandingkan **Menggunakan 3-Layer**, siapa yang lebih unggul pada masing-masing unsur, beserta alasan lengkapnya:

No	Unsur yang Diuji Coba	Tanpa Layer (Non-Layered)	Menggunakan 3-Layer	Lebih Unggul	Alasan Mengapa Lebih Unggul
1	**Struktur Pemisahan File & Folder (Hubungan HTML, CSS, & PHP)**	Belum memiliki pemisahan tanggung jawab layer yang jelas (bisa 1 atau beberapa file bercampur; pada studi kasus ini di 1 file index.php).	Dipisahkan tegas ke 3 folder: presentation/ (HTML/CSS/JS), business-logic/ (PHP API), & data-access/ (SQL).	**3-Layer Architecture**	Kode jauh lebih rapi dan terorganisasi karena setiap file/folder memiliki satu tanggung jawab layer yang tegas.
2	**Kecepatan Pembuatan Awal (Initial Development Speed)**	Sangat cepat dibuat di awal karena kode HTML, PHP, dan SQL bisa langsung ditulis menyatu tanpa menyiapkan REST API.	Membutuhkan waktu awal lebih lama karena harus membuat kontrak JSON, Service, dan Repository di 3 folder.	**Non-Layered Architecture**	Untuk aplikasi yang sangat kecil/sederhana, menulis langsung tanpa pemisahan layer lebih praktis dan cepat.
3	**Pengerjaan secara Individu vs Tim (Teamwork & Kolaborasi)**	Cocok untuk 1 orang (individu), tetapi sulit bagi tim karena kode tampilan, logika, & SQL belum dipisah secara tegas (di sini menyatu di index.php).	Sangat ideal untuk kerja tim: anggota A mengerjakan folder presentation/, B di business-logic/, C di data-access/.	**3-Layer Architecture**	Pembagian tugas kelompok menjadi paralel sesuai layer dan tidak saling menimpa kode (bebas merge conflict).
4	**Kemudahan Penelusuran Kesalahan (Debugging / Error Tracing)**	Sulit ditelusuri saat kode membesar karena harus mencari apakah error berasal dari HTML, validasi, atau SQL di 1 file.	Sangat mudah dilacak: error tampilan cek di presentation/, error validasi di CustomerService, error SQL di Repository.	**3-Layer Architecture**	Sumber bug langsung terisolasi per folder layer sehingga proses perbaikan kesalahan jauh lebih cepat.
5	**Kecepatan & Kenyamanan Interaksi (Pengalaman Pengguna / UX)**	Setiap kali tambah/cari/hapus data, browser harus memuat ulang (reload) seluruh halaman HTML dari awal.	Lebih cepat & responsif karena hanya mengirim/menerima data JSON via fetch() tanpa me-reload seluruh halaman.	**3-Layer Architecture**	Pertukaran data menggunakan REST API & JSON lebih ringan dibandingkan me-render ulang seluruh halaman HTML.
6	**Kemudahan Pemeliharaan Desain (Maintainability UI vs Database)**	Berisiko tinggi: saat mengubah desain tabel HTML/CSS, programmer bisa tidak sengaja merusak kode query SQL di atasnya.	Sangat aman: perubahan desain di index.html / style.css sama sekali tidak menyentuh file PHP maupun query SQL.	**3-Layer Architecture**	Perubahan pada tampilan antarmuka tidak memengaruhi logika bisnis maupun struktur akses database.
7	**Kemudahan Pengembangan Fitur (Scalability / Fitur Bertambah)**	Semakin banyak fitur ditambahkan, file index.php semakin panjang dan rumit (menjadi spaghetti code).	Sangat mudah dikembangkan: penambahan fitur baru cukup menambah fungsi secara modular di masing-masing layer.	**3-Layer Architecture**	Struktur modular menjaga kode tetap bersih meskipun fitur aplikasi terus bertambah besar.
8	**Keamanan & Penggunaan Ulang (Security & Reusability API)**	Query database berada di file tampilan dan tidak bisa dipakai oleh aplikasi Mobile (karena outputnya HTML).	Database terisolasi di Data Layer, dan endpoint REST API JSON (customers.php) bisa langsung dipakai Web & Mobile.	**3-Layer Architecture**	Satu Backend REST API JSON dapat melayani berbagai platform sekaligus (Web maupun Android/iOS).

Tabel 7.1 Tabel Utama Hasil Pengujian dan Perbandingan Non-Layered vs 3-Layer Architecture

BAB 8. JAWABAN 5 PERTANYAAN DISKUSI DAN KESIMPULAN

8.1 Jawaban Lengkap 5 Pertanyaan Diskusi (Halaman 17 PDF)

1. Mana yang lebih mudah dibuat?

Non-Layered Architecture lebih mudah dan cepat dibuat di awal (khususnya jika dikerjakan sendiri untuk aplikasi berskala kecil), karena programmer cukup menulis koneksi database, validasi PHP, query SQL, dan tampilan HTML di dalam satu file index.php tanpa perlu membuat kontrak REST API JSON maupun membagi kode ke dalam beberapa folder.

2. Mana yang lebih mudah diperbaiki ketika terjadi kesalahan (bug)?

3-Layer Architecture jauh lebih mudah diperbaiki dan ditelusuri. Karena adanya pemisahan folder per layer secara tegas, penelusuran kesalahan (*debugging*) langsung terarah: jika terjadi kesalahan tampilan maka diperiksa di folder presentation/, jika aturan validasi keliru diperiksa di CustomerService.php (folder business-logic/), dan jika query SQL bermasalah diperiksa di CustomerRepository.php (folder data-access/).

3. Mana yang lebih mudah dikembangkan ketika fitur bertambah?

3-Layer Architecture jauh lebih mudah dikembangkan. Penambahan fitur baru maupun pembagian tugas di dalam tim dapat dilakukan secara paralel per folder layer tanpa saling merusak kode anggota lain. Selain itu, endpoint REST API JSON (api/customers.php) dapat langsung digunakan ulang oleh aplikasi Mobile (Android/iOS) tanpa menulis ulang logika bisnis.

4. Apa risiko jika seluruh kode ditempatkan dalam satu file?

Risiko utamanya adalah kode menjadi sangat panjang dan tercampur aduk (*spaghetti code*), perubahan sederhana pada desain HTML/CSS berisiko tinggi merusak logika validasi PHP atau query SQL di atasnya, sangat sulit dikerjakan bersama oleh tim karena rawan bentrok (*merge conflict*) pada satu file yang sama, serta sulit diuji secara terpisah.

5. Apakah aplikasi tanpa 3-layer selalu lebih buruk?

Tidak selalu. Untuk aplikasi yang sangat sederhana, skrip satu halaman, prototipe cepat (*MVP*), atau tugas individu berskala kecil, pendekatan tanpa layer justru praktis dan hemat waktu. Namun, untuk aplikasi perusahaan yang dikerjakan secara tim, membutuhkan pemeliharaan jangka panjang, dan terus berkembang, **3-Layer Architecture mutlak lebih unggul**.

8.2 Kesimpulan Akhir

Berdasarkan seluruh analisis teori dan hasil uji coba implementasi studi kasus **Customer Management System**, dapat disimpulkan bahwa **Non-Layered Architecture** hanya unggul dari segi kecepatan pembuatan awal untuk aplikasi kecil yang dikerjakan oleh satu orang. Sebaliknya, pada 7 dari 8 unsur pengujian lainnya—meliputi **kerapian pemisahan file/folder**, **kemudahan kerja sama tim**, **kemudahan penelusuran kesalahan (debugging)**, **kecepatan respons antarmuka tanpa reload**, **keamanan modifikasi desain**, **kemudahan penambahan fitur baru**, serta **reusabilitas REST API JSON**—pendekatan **3-Layer Web Application Architecture** terbukti jauh lebih unggul.